Lab 7: Fuzzing for Input Validation Bugs

Project: **L6_L7_34 - Tennis Scoring** (repo: SETAPESU26/L6_L7_34_tennis_scoring, folder `lab7`)

Name: Shubham D Bhat SRN: PES1UG24AM273

Environment: Python 3.12.3, Hypothesis 6.170.15, pytest 9.2.0. Note: the handout names the files `processor.py` / `test_processor.py`; in this repo they are `tennis_scoring.py` / `test_tennis_scoring.py`.

Summary

Stage	Result
Original code vs. Hypothesis tests	**10 failed, 4 passed** - all three functions crash on empty strings, None, a lone '-', unknown tokens and non-string input
Fixed code vs. same tests	**14 passed** (also 14 passed with 5,000 generated examples per test)

Step 1 - Install dependencies

Hypothesis and pytest were installed with pip (shown here in a clean virtual environment).

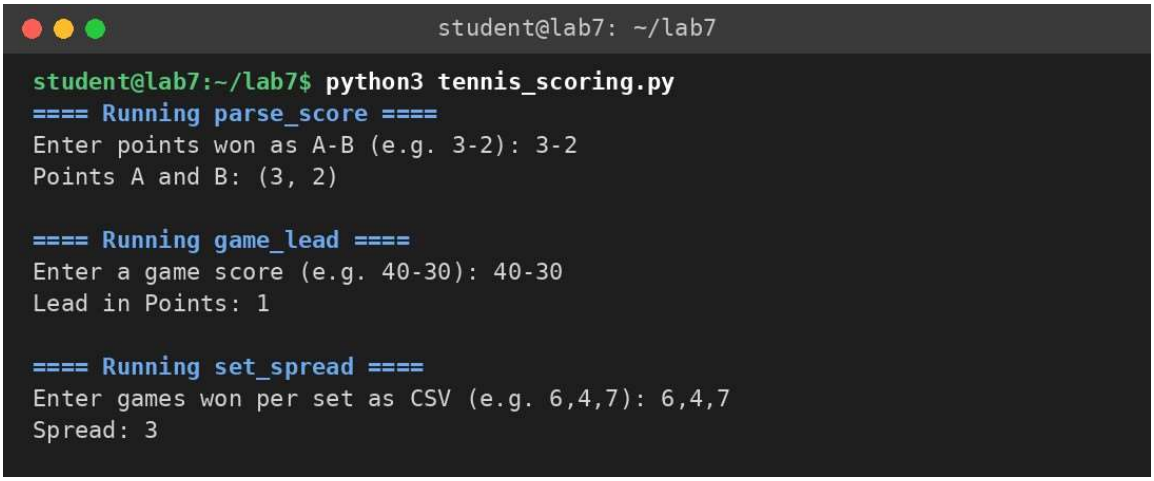
```
student@lab7: ~/lab7
student@lab7:~/lab7$ pip install hypothesis && pip install pytest
Collecting hypothesis
Collecting sortedcontainers<3.0.0,>=2.1.0 (from hypothesis)
Using cached hypothesis-6.170.15-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (1.2 MB)
Using cached sortedcontainers-2.4.0-py2.py3-none-any.whl (29 kB)
Installing collected packages: sortedcontainers, hypothesis
Successfully installed hypothesis-6.170.15 sortedcontainers-2.4.0

Collecting pytest
Collecting iniconfig>=2 (from pytest)
Collecting packaging>=24 (from pytest)
Collecting pluggy<2,>=1.6 (from pytest)
Collecting pygments>=2.15 (from pytest)
Using cached pytest-9.2.0-py3-none-any.whl (411 kB)
Using cached iniconfig-2.3.1-py3-none-any.whl (7.6 kB)
Using cached pluggy-1.7.0-py3-none-any.whl (27 kB)
Installing collected packages: pygments, pluggy, packaging, iniconfig, pytest
Successfully installed iniconfig-2.3.1 packaging-26.3 pluggy-1.7.0 pygments-2.21.0 pytest-9.2.0
```

Step 1: installing hypothesis and pytest.

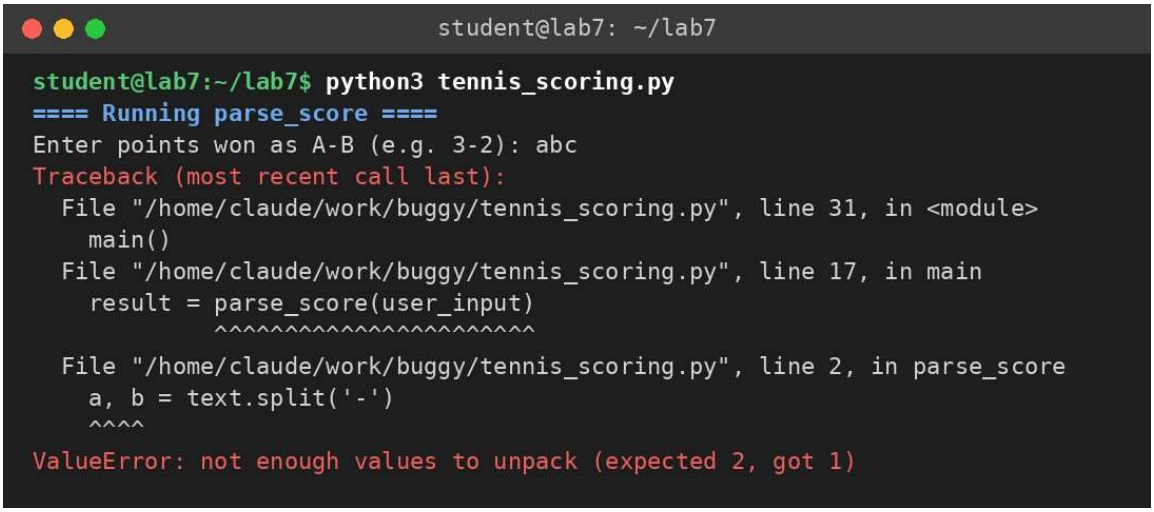
Step 2 - Run the original (buggy) application

With well-formed input the program works, so the bugs are invisible to a casual run.

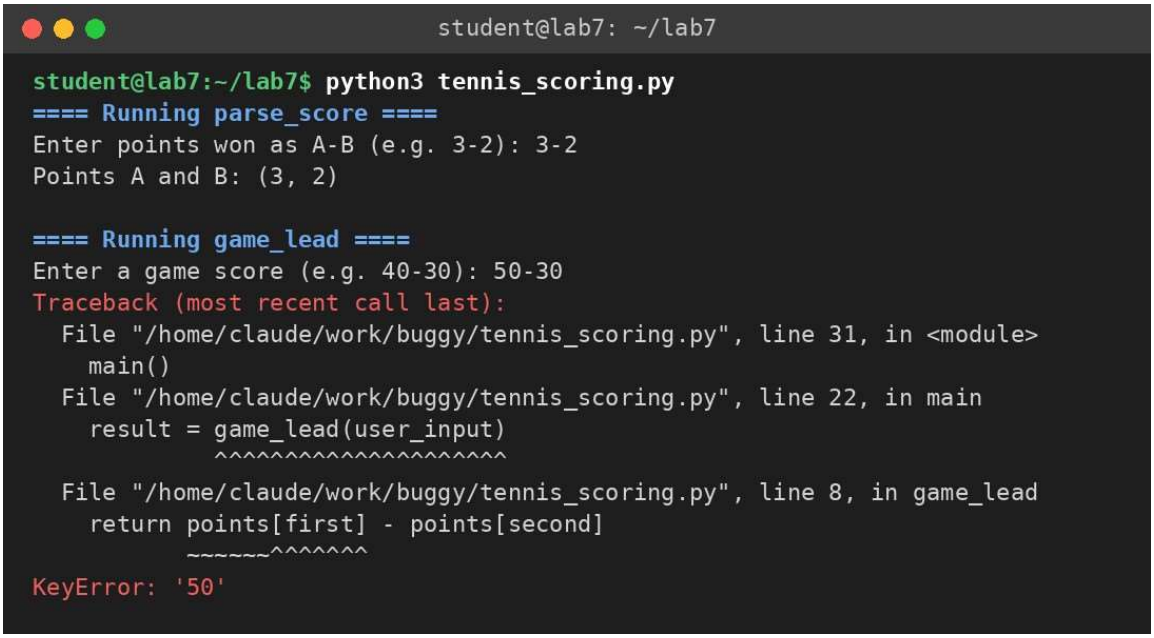


Step 2a: buggy app, valid input - works.

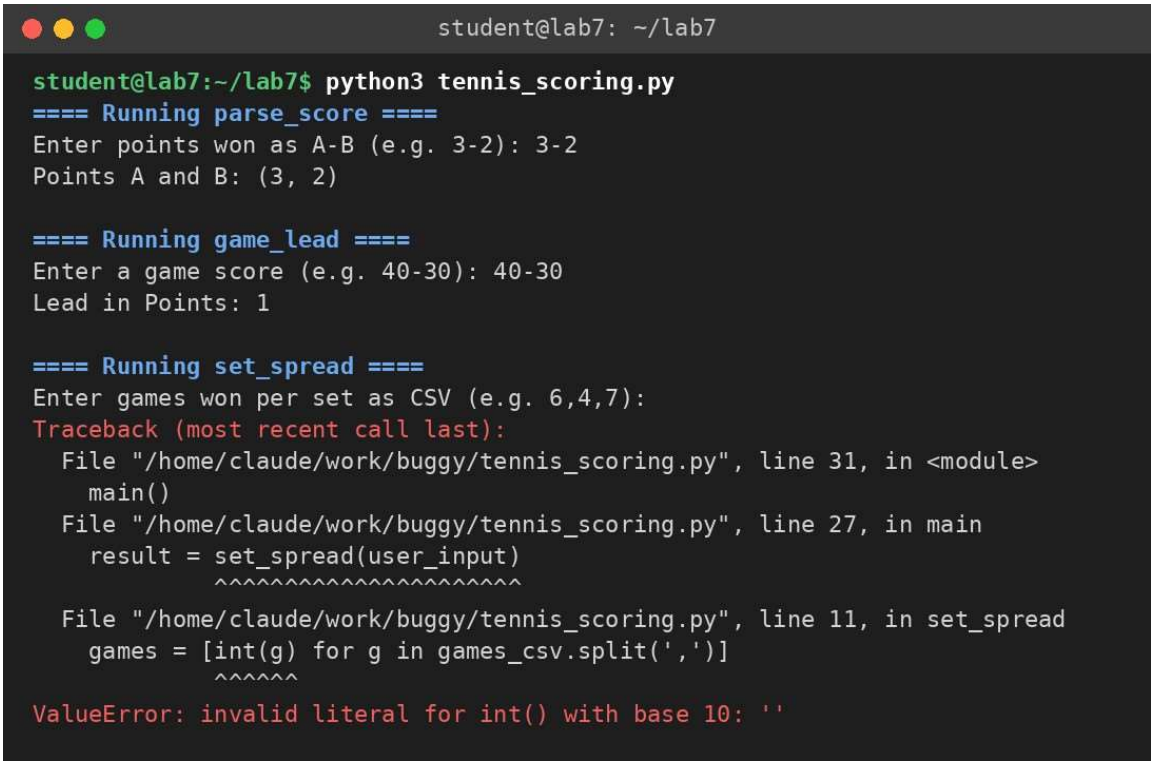
With invalid input each function crashes with a raw traceback - one for each of the three functions:



Step 2b: parse_score('abc') - ValueError.



Step 2c: game_lead('50-30') - KeyError.



Step 2d: set_spread('') - ValueError.

Step 3 - Complete test_tennis_scoring.py with Hypothesis

The three required no-crash tests were completed exactly as the handout pattern describes: Hypothesis generates arbitrary text or None and the test fails if the function raises anything.

```
@given(st.text() | st.none())
def test_parse_score_no_crash(s):
    try:
        parse_score(s)
    except Exception as e:
        assert False, f"parse_score crashed with: {e!r}"
# ...identical pattern for game_lead and set_spread
```

To "uncover more edge cases automatically for all three functions", 11 further tests were added (14 in total):

Test group	Tests	What it checks

No-crash (required)	`test_parse_score_no_crash`
	`test_game_lead_no_crash`
	`test_set_spread_no_crash`
		Any text or None must never raise an exception.

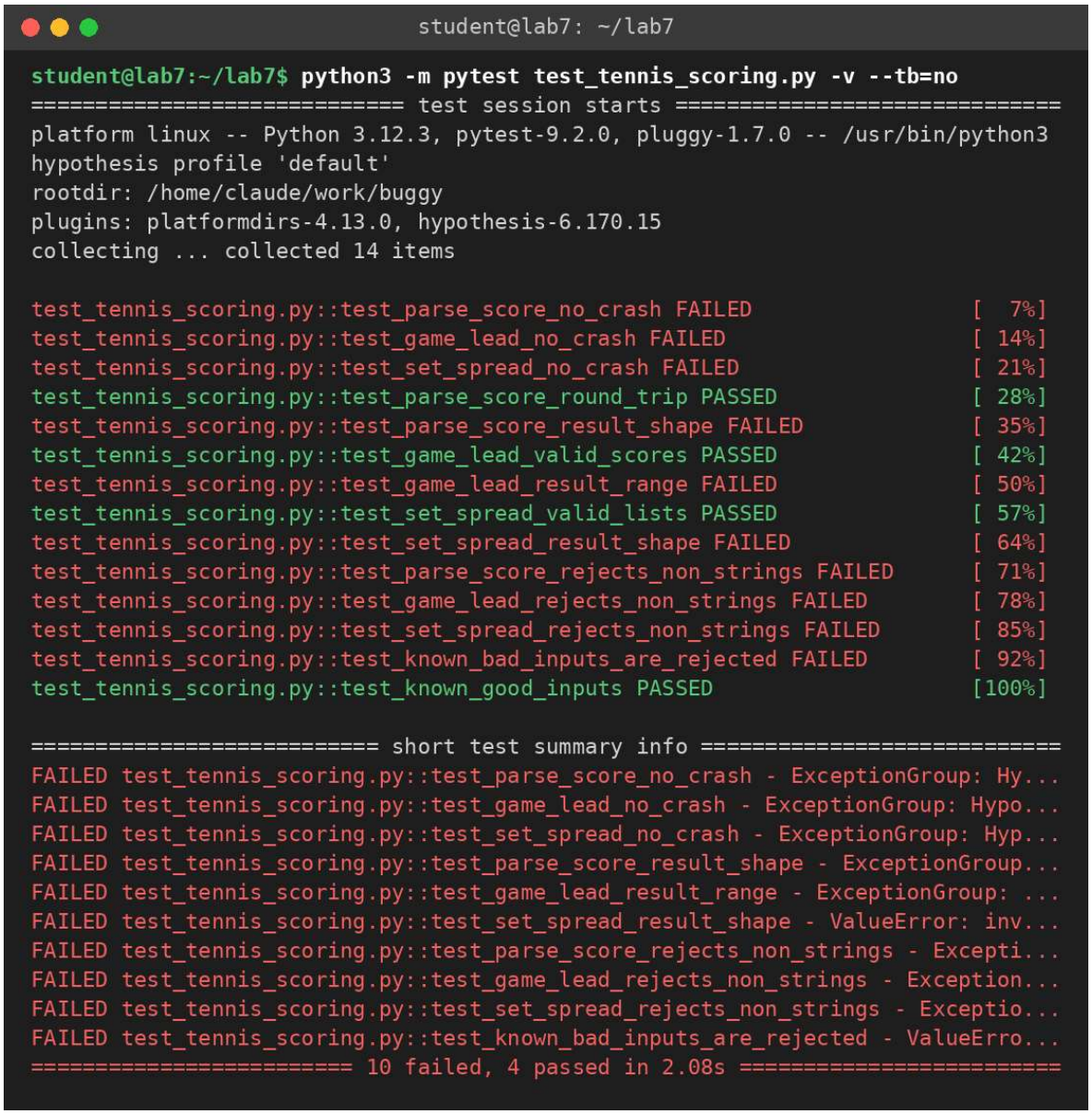
Correctness on valid input	`test_parse_score_round_trip`
	`test_game_lead_valid_scores`
	`test_set_spread_valid_lists`
		f"{a}-{b}" parses back to (a, b); every legal tennis pair gives the right lead and swapping players flips the sign; any list of non-negative ints gives max - min.

Output shape on any text	`test_parse_score_result_shape`
	`test_game_lead_result_range`
	`test_set_spread_result_shape`
		Result is either None (rejected) or a value of the right type/range (tuple of two non-negative ints; int in -3..3; int >= 0).

Malformed types	`test_*_rejects_non_strings (x3)`	ints, floats (incl. NaN/inf), bools, bytes, lists, dicts and tuples are rejected with None - no TypeError/AttributeError.
Regression	`test_known_bad_inputs_are_rejected` `test_known_good_inputs`	Concrete cases: '', '-', '3--2', '1,,3', '6,4,', '1_0-2', Arabic-Indic digits, superscript digits, '+3-2' must be rejected; '3-2', ' 3 - 2 ', '40-30', '6,4,7' must still work.

Step 4 - Run the tests against the original code (failing)

Result: **10 failed, 4 passed**. The 4 passes are the "valid input" tests - the original code is fine on good input and breaks only on edge cases.



Step 4a: per-test status and summary (10 failed, 4 passed).

Failure details for the three required no-crash tests (full default pytest output; Hypothesis reports each distinct bug with its minimal "Failing test case"):

```
student@lab7: ~/lab7
student@lab7:~/lab7$ python3 -m pytest test_tennis_scoring.py -k no_crash
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.2.0, pluggy-1.7.0
rootdir: /home/claude/work/buggy
plugins: platformdirs-4.13.0, hypothesis-6.170.15
collected 14 items / 11 deselected / 3 selected

test_tennis_scoring.py FFF                                               [100%]

=================================== FAILURES ===================================
_____________________ test_parse_score_no_crash _____________________
  + Exception Group Traceback (most recent call last):
  |   File "/home/claude/work/buggy/test_tennis_scoring.py", line 35, in test_parse_score_no_crash
  |     def test_parse_score_no_crash(s):
  |                    ^^^
  | ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "/home/claude/work/buggy/test_tennis_scoring.py", line 37, in test_parse_score_no_crash
    |     parse_score(s)
    |   File "/home/claude/work/buggy/tennis_scoring.py", line 3, in parse_score
    |     return int(a), int(b)
    |            ^^^^^^
    | ValueError: invalid literal for int() with base 10: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/home/claude/work/buggy/test_tennis_scoring.py", line 39, in test_parse_score_no_crash
    |     assert False, f"parse_score crashed with: {e!r}"
    | AssertionError: parse_score crashed with: ValueError("invalid literal for int() with base 10: ''")
    | assert False
    | Failing test case: test_parse_score_no_crash(
    |     s='-',
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "/home/claude/work/buggy/test_tennis_scoring.py", line 37, in test_parse_score_no_crash
    |     parse_score(s)
    |   File "/home/claude/work/buggy/tennis_scoring.py", line 2, in parse_score
    |     a, b = text.split('-')
    |     ^^^^
    | ValueError: not enough values to unpack (expected 2, got 1)
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/home/claude/work/buggy/test_tennis_scoring.py", line 39, in test_parse_score_no_crash
    |     assert False, f"parse_score crashed with: {e!r}"
    | AssertionError: parse_score crashed with: ValueError('not enough values to unpack (expected 2, got 1)')
    | assert False
    | Failing test case: test_parse_score_no_crash(
    |     s='',  # or any other generated value
    | )
    +---------------- 3 ----------------
    | Traceback (most recent call last):
    |   File "/home/claude/work/buggy/test_tennis_scoring.py", line 37, in test_parse_score_no_crash
... (continued on next image)
```

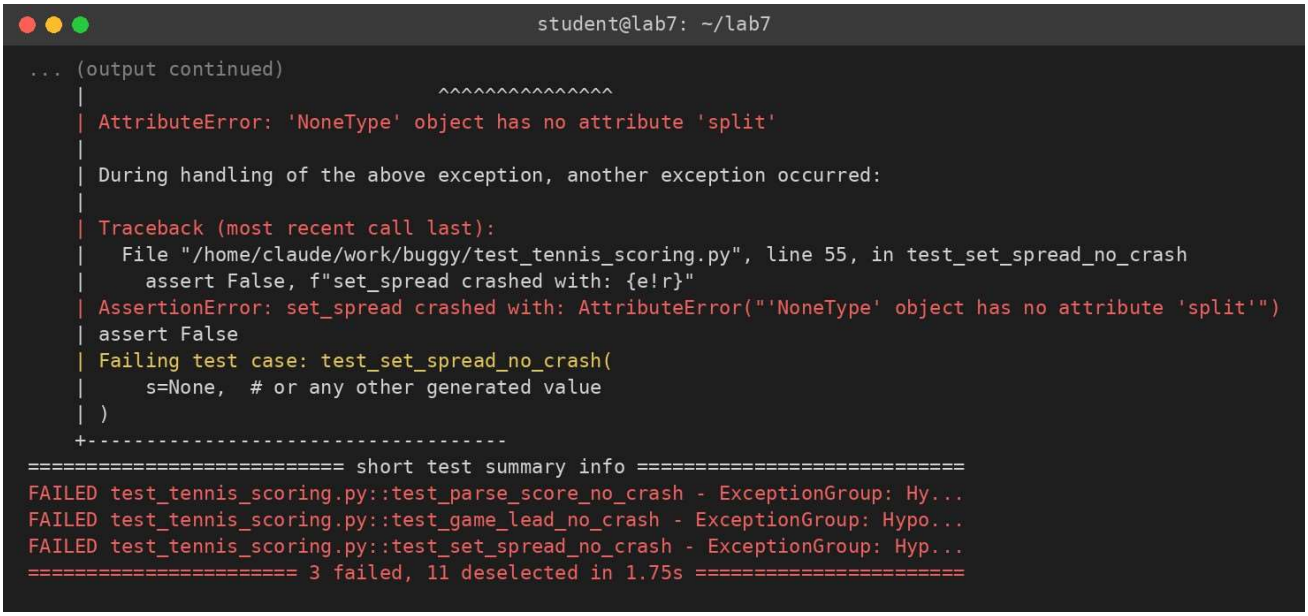
Step 4b: failing no-crash tests with Hypothesis counter-examples.



Step 4b: failing no-crash tests with Hypothesis counter-examples. (part 2 of 4)



Step 4b: failing no-crash tests with Hypothesis counter-examples. (part 3 of 4)



Step 4b: failing no-crash tests with Hypothesis counter-examples. (part 4 of 4)

Bugs discovered

Function	Hypothesis counter-example	Exception	Root cause
`parse_score`	`s = None`	AttributeError: 'NoneType' has no attribute 'split'	No type check on input.
	`s = ''`	ValueError: not enough values to unpack	Assumes the text always contains exactly one '-'.
	`s = '-'`	ValueError: invalid literal for int() with base 10: ''	Empty operand passed straight to int().
	`x = 0, x = b''`	AttributeError / TypeError	Non-string input is not rejected.
`game_lead`	`s = None`	AttributeError	No type check on input.
	`s = ''`	ValueError: not enough values to unpack	Assumes exactly one '-'.
	`s = '-'`	KeyError: ''	Dictionary lookup of an unvalidated token.
	`'50-30'` (found in Step 2)	KeyError: '50'	Only 0/15/30/40 are valid; no whitelist check.
	`x = 0, x = b''`	AttributeError / TypeError	Non-string input is not rejected.
`set_spread`	`s = None`	AttributeError	No type check on input.
	`s = ''`	ValueError: invalid literal for int(): ''	Empty CSV / empty element ('1,,3', '6,4,') not handled.
	`x = 0, x = b''`	AttributeError / TypeError	Non-string input is not rejected.

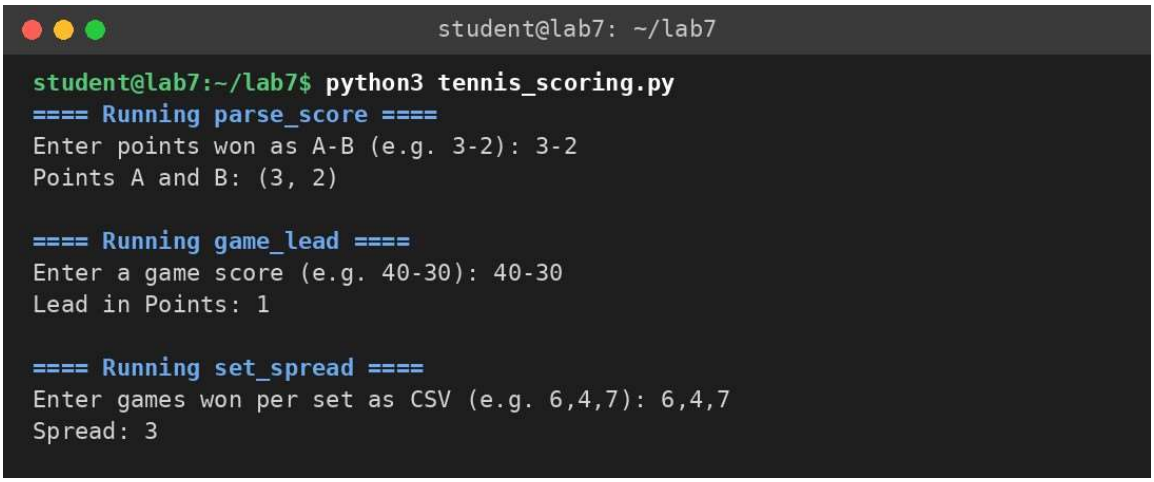
Silent-acceptance bugs (found by code review, locked in by the regression test). Python's `int()` also accepts '1_0' (= 10), '+3', and non-ASCII digits such as the Arabic-Indic digit three (U+0663, = 3). The original code would happily treat those as valid scores; the fixed code does not.

Step 5 - Fix the application

Design decision: the handout's no-crash property means the functions must never raise. Each function now validates its input and returns `None` when the input is invalid; `main()` turns `None` into a friendly message.

Weakness	Defensive fix applied
Non-string / None input	`isinstance(value, str)` check at the top of every function.
Wrong number of '-' separators	`text.split('-')` result must have exactly 2 parts, otherwise return None.
Bad numbers (empty, 'abc', '1.5', '+3', '1_0', Unicode digits)	New helper `_to_count()` accepts only ASCII digits via `re.fullmatch(r'[0-9]+')` (after trimming spaces), then calls int(). Negative values are rejected.
Unknown tennis point value (KeyError)	Whitelist: both sides must be in `POINTS = {'0','15','30','40'}`.
Empty elements in the CSV	Every element is converted first; if any is None the whole input is rejected before max()/min().
Huge input (DoS / digit-limit ValueError)	`MAX_INPUT_LENGTH = 1000` characters; longer input is rejected.
main() crashing / EOF	Prints "Invalid input: ..." when a function returns None; catches EOFError and KeyboardInterrupt.

Valid input behaves exactly as before:



Step 5a: fixed app, valid input (same results as the original).

The inputs that previously produced tracebacks are now rejected cleanly:

```
student@lab7: ~/lab7
student@lab7:~/lab7$ python3 tennis_scoring.py
==== Running parse_score ====
Enter points won as A-B (e.g. 3-2): abc
Invalid input: expected two non-negative whole numbers as A-B.

==== Running game_lead ====
Enter a game score (e.g. 40-30): 50-30
Invalid input: expected two of 0, 15, 30, 40 as A-B.

==== Running set_spread ====
Enter games won per set as CSV (e.g. 6,4,7):
Invalid input: expected comma-separated whole numbers.
```

Step 5b: fixed app with 'abc', '50-30' and an empty line - no crash.

Whitespace is tolerated, and an invalid CSV is rejected while the other functions still work:

```
student@lab7: ~/lab7
student@lab7:~/lab7$ python3 tennis_scoring.py
==== Running parse_score ====
Enter points won as A-B (e.g. 3-2):  3 - 2
Points A and B: (3, 2)

==== Running game_lead ====
Enter a game score (e.g. 40-30): 0-40
Lead in Points: -3

==== Running set_spread ====
Enter games won per set as CSV (e.g. 6,4,7): 6,,4
Invalid input: expected comma-separated whole numbers.
```

Step 5c: fixed app with ' 3 - 2 ', '0-40' and '6,,4'.

Step 6 - Re-run the tests against the fixed code

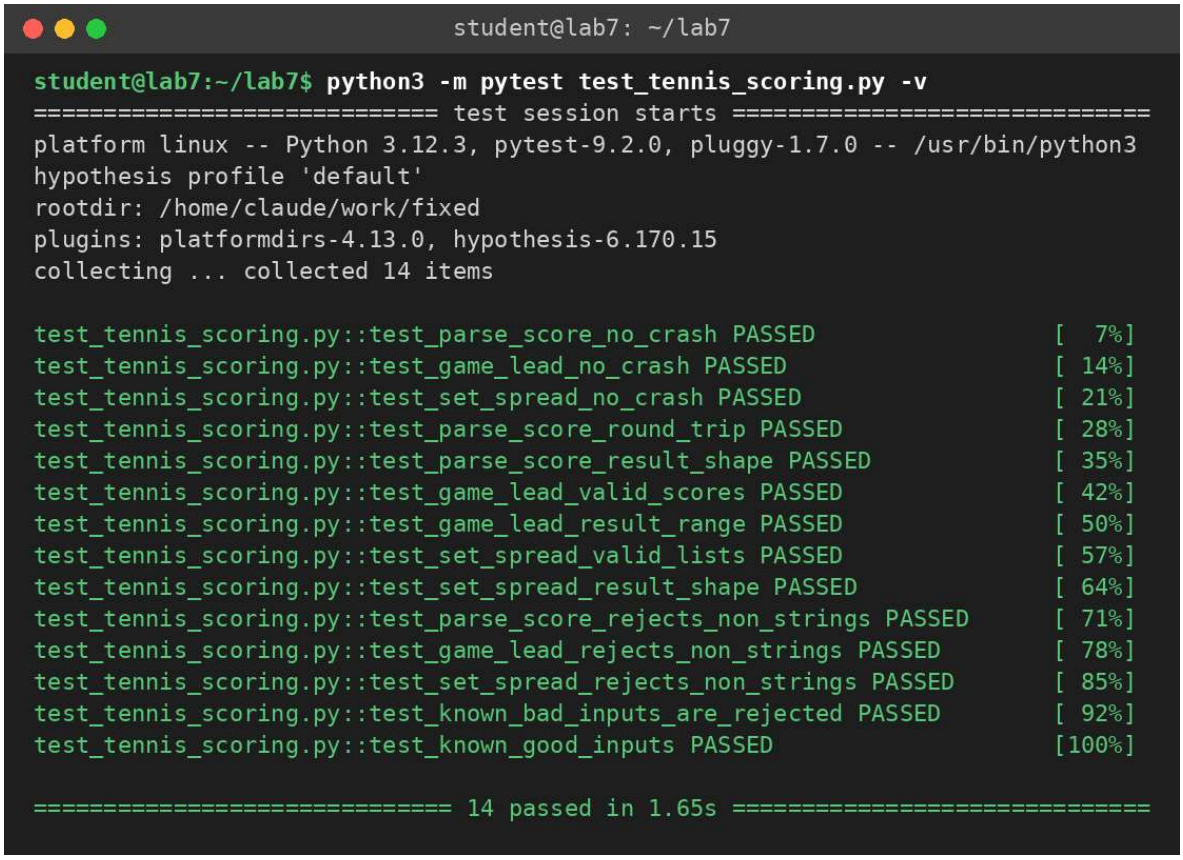
The same, unchanged test file now passes completely: **14 passed**.

```
student@lab7: ~/lab7
student@lab7:~/lab7$ python3 -m pytest test_tennis_scoring.py
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.2.0, pluggy-1.7.0
rootdir: /home/claude/work/fixed
plugins: platformdirs-4.13.0, hypothesis-6.170.15
collected 14 items

test_tennis_scoring.py ..............                                    [100%]

============================== 14 passed in 1.59s ==============================
```

Step 6a: pytest on the fixed code - 14 passed.



Step 6b: verbose view - every test PASSED.

As an extra check, Hypothesis was run with `max_examples=5000` per test (some strategies, e.g. the 16 valid tennis point pairs, are exhausted sooner): **14 passed in 21.68 s**.

Conclusion

Fuzzing found crashes in all three functions in about two seconds of test time and shrank each one to a minimal input (`''`, `'-'`, `None`, `0`, `b''`). After switching to validate-then-convert, every generated input is either processed correctly or rejected with `None`, and the full suite passes.